Deadlock

Deadlock

- Two or more goroutines are waiting forever for each other to release resources, so none of them can proceed.
- Runtime error: `fatal error: all goroutines are asleep - deadlock!`
- Types of Deadlock in Go:
  - *Channel Deadlock*
  - *Goroutine Circular Dependency*
  - *Mutex Deadlock*
  - *Multiple Locks (Important for Interviews)*

Deadlock Types: Channel Deadlock

- No Sender
- No Receiver
- Buffered Channel is full

Solution:

- Make sure sender or receiver exists.
- No Buffer overflow

Deadlock Types: Goroutine Circular Dependency

Problem:

Goroutine 1 waits on ch2

Goroutine 2 waits on ch1

→ circular wait → deadlock

Solution:

Initiate a value to break the deadlock

```go
func main() {
    ch1 := make(chan int)
    ch2 := make(chan int)

    go func() {
        ch1 <- <-ch2
    }()

    go func() {
        ch2 <- <-ch1
    }()

    select {}
}
```

Deadlock Types: Mutex Deadlock

- Mutex is not re-entrant
- Same goroutine cannot lock twice

Solution:

Never Lock Twice; use defer mu.Unlock()

```go
var mu sync.Mutex

func main() {
    mu.Lock()
    mu.Lock() // deadlock (same goroutine locking twice)
}
```

Deadlock Types: Multiple Locks

Issue: Lock Ordering Issue

Step-by-step Execution

1. Goroutine 1
   a. *Locks mu1*
   b. *Tries to lock mu2 → waits*
2. Goroutine 2
   a. *Locks mu2*
   b. *Tries to lock mu1 → waits*

Result:

- G1 holds mu1, waiting for mu2
- G2 holds mu2, waiting for mu1
- → circular dependency → deadlock

Solution:

ALWAYS lock mu1 → mu2 everywhere

```go
var mu1, mu2 sync.Mutex

func main() {
    go func() {
        mu1.Lock()
        mu2.Lock()
    }()

    go func() {
        mu2.Lock()
        mu1.Lock()
    }()

    select {}
}
```

Deadlock or Not?

```go
func main() {
    ch := make(chan int)

    go func() {
        ch <- 1
        ch <- 2
    }()

    fmt.Println(<-ch)
}
```

Deadlock or Not?

Answer: Deadlock

Why?

- Main goroutine:
- Receives only once → fmt.Println(<-ch)
- Worker goroutine:
- Sends twice

```go
func main() {
    ch := make(chan int)

    go func() {
        ch <- 1
        ch <- 2
    }()

    fmt.Println(<-ch)
}
```

Deadlock or Not?

```go
func main() {
    ch1 := make(chan int)
    ch2 := make(chan int)

    go func() {
        ch1 <- 1
        fmt.Println(<-ch2)
    }()

    go func() {
        ch2 <- 2
        fmt.Println(<-ch1)
    }()

    select {}
}
```

Deadlock or Not?

Answer: No Deadlock

Why?

Execution flow:

G1 sends to ch1

G2 sends to ch2

```go
func main() {
    ch1 := make(chan int)
    ch2 := make(chan int)

    go func() {
        ch1 <- 1
        fmt.Println(<-ch2)
    }()

    go func() {
        ch2 <- 2
        fmt.Println(<-ch1)
    }()

    select {}
}
```

Deadlock or Not?

```go
var mu sync.Mutex

func main() {
    mu.Lock()

    go func() {
        mu.Lock()
    }()

    time.Sleep(time.Second)
}
```

Deadlock or Not?

Answer: Deadlock (logical)

```go
var mu sync.Mutex

func main() {
    mu.Lock()

    go func() {
        mu.Lock()
    }()

    time.Sleep(time.Second)
}
```

Deadlock or Not?

```go
func main() {
    ch := make(chan int)

    go func() {
        fmt.Println(<-ch)
    }()

    ch <- 10
}
```

Deadlock or Not?

Answer: No Deadlock

Why?

Goroutine starts and waits for receive

Main sends value → matched

```go
func main() {
    ch := make(chan int)

    go func() {
        fmt.Println(<-ch)
    }()

    ch <- 10
}
```

Deadlock or Not?

```go
var wg sync.WaitGroup

func main() {
    wg.Add(1)

    go func() {
        // forgot wg.Done()
    }()

    wg.Wait()
}
```

Deadlock or Not?

Answer: Deadlock

```go
var wg sync.WaitGroup

func main() {
    wg.Add(1)

    go func() {
        // forgot wg.Done()
    }()

    wg.Wait()
}
```

Deadlock or Not?

```go
func main() {
    a := make(chan int)
    b := make(chan int)

    go func() {
        a <- 1
        <-b
    }()

    go func() {
        b <- 2
        <-a
    }()

    select {}
}
```

Deadlock or Not?

Answer: No Deadlock

Why?

Execution:

G1 sends to a

G2 sends to b

```go
func main() {
    a := make(chan int)
    b := make(chan int)

    go func() {
        a <- 1
        <-b
    }()

    go func() {
        b <- 2
        <-a
    }()

    select {}
}
```

Deadlock Conditions

4 Necessary Conditions for Deadlock:

1. Mutual Exclusion → resource locked (mutex)
2. Hold and Wait → holding one, waiting for another
3. No Preemption → cannot forcibly take resource
4. Circular Wait → cycle exists

If all 4 exist → deadlock possible